



Sistemas Distribuidos 2027-1 • Grupo 2 • Sesión 4

Anatomía de un sistema real, segunda parte

Las seis piezas, los datos y en qué idioma se hablan los servicios

Sábado 12 de septiembre de 2026 • Prof. Oscar Ruiz • FI-UNAM

El mapa de la sesión: 120 minutos, una decisión al final

B

Cómo se hablan: el cierre

puede, gRPC, REST o gRPC

20 min

O

La pregunta de la semana

12-factor, factor III

10 min

C

Las seis piezas

gateway a observabilidad

57 min

E

Los datos

políglota, familias

10 min

R

Recapitulación

el mapa completo

4 min

F

La decisión de hoy • 09:44

FarmaYa: ¿reintentar o no?

12 min

G

Para la S5

Raft y compose

4 min



Hoy no hay demo: el sistema corriendo se desglosa en el lab de la S5.

Lo que quedó de la S3, en cuatro frases

1 Timeout

No dice qué pasó: solo que dejaste de esperar.

```
2015 ms: timeout  
5000 ms: cobro
```

2 Exactly-once

No se entrega: at-least-once más deduplicación.

```
at-least-once + dedup  
Dos Generales, TCP, Kafka
```

3 REST

La red explícita: recursos, verbos con contrato.

```
seguro ≠ idempotente  
RFC 9110, nivel 2
```

4 Idempotencia

Clave por intención más reserva atómica fuera del proceso.

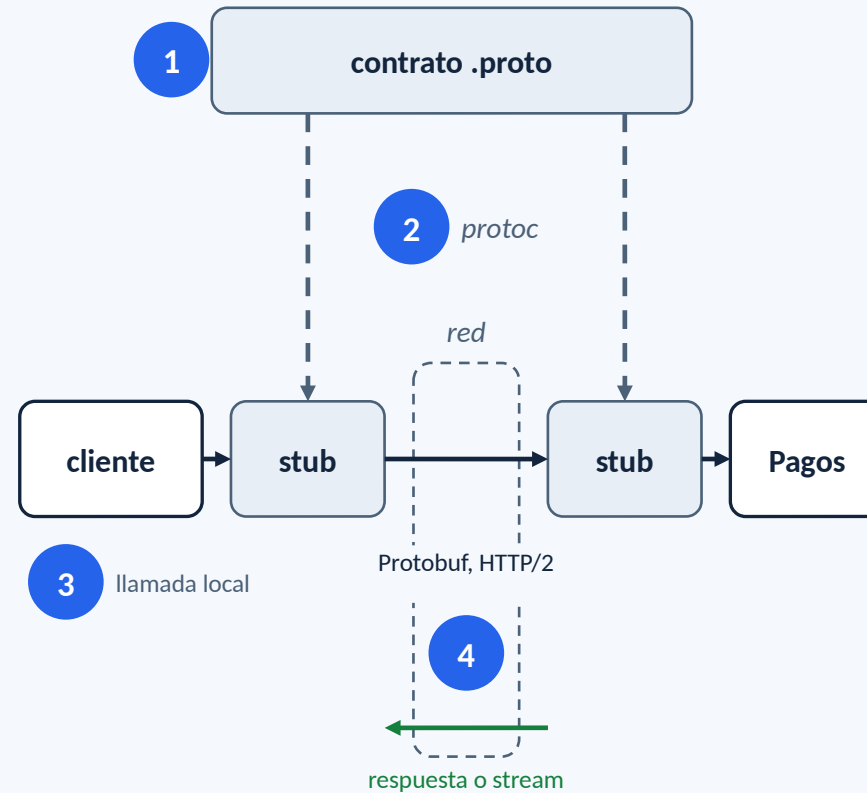
```
Idempotency-Key: 24 h, 409  
UNIQUE, no un Map
```



Tarea opcional: [s03/opcional-idempotencia/](#)

gRPC: RPC vuelve, con contrato y con deadline

```
service Pagos {  
  rpc Autorizar(Cargo) returns (Resultado);  
}  
  
message Cargo {  
  string cuenta = 1;  
  double monto = 2;  
}
```



38 → 9 bytes

```
{"id":42,"nombre":"Ada","activo":true}
```

```
08 2A 12 03 41 64 61 18 01
```

sin nombres de campo

deadline

en cada llamada: la lección de 1984

26-feb-2015

Google, HTTP/2, streaming nativo, grpcurl

REST o gRPC: mismo problema, trade-offs distintos

	REST	gRPC
Formato en el cable	JSON, texto	Protobuf, binario
Contrato	opcional (OpenAPI)	obligatorio (.proto)
Transporte	HTTP/1.1 o superior	HTTP/2
Streaming	limitado	nativo, bidireccional
Depuración	curl y navegador, legible	herramientas dedicadas (grpcurl)
Clientes	cualquier cliente HTTP	código generado por lenguaje
Uso dominante	APIs públicas, el borde	interno, servicio a servicio



La frontera pública favorece REST

Cualquier cliente lo lee; el costo en bytes se paga una vez, en el borde.



El tráfico interno favorece gRPC

Miles de llamadas: el ahorro se multiplica y el contrato evita divergencias.

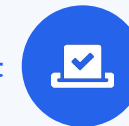
No es una regla: es un trade-off.



Mexi Banco: gateway → servicios es REST; transferencias → cuentas, el candidato a gRPC. Aquí gRPC es criterio, no lab.

Pregunta de la semana: ¿dónde vive la configuración?

vota en el chat



Según The Twelve-Factor App, la Lectura 3:

A



un archivo de propiedades
en la imagen

B



variables de entorno, fuera
del código

C



una tabla en la base de
datos

D



el código, con un perfil por
entorno

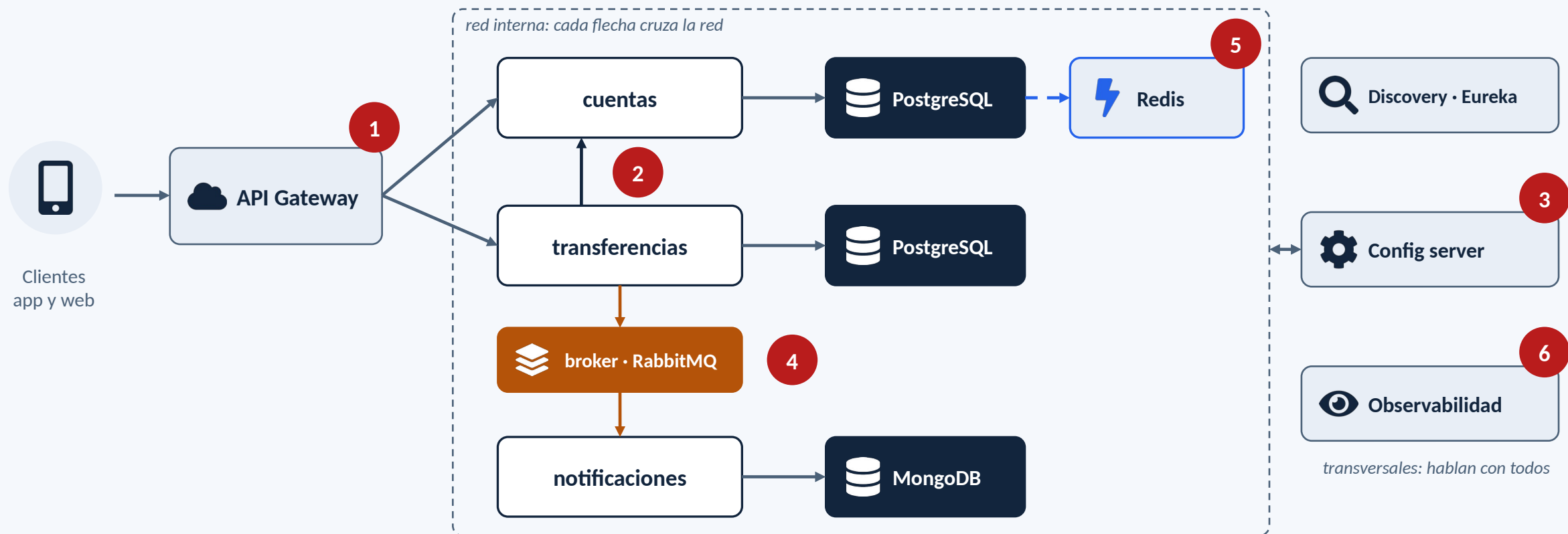


Prueba de fuego del factor III

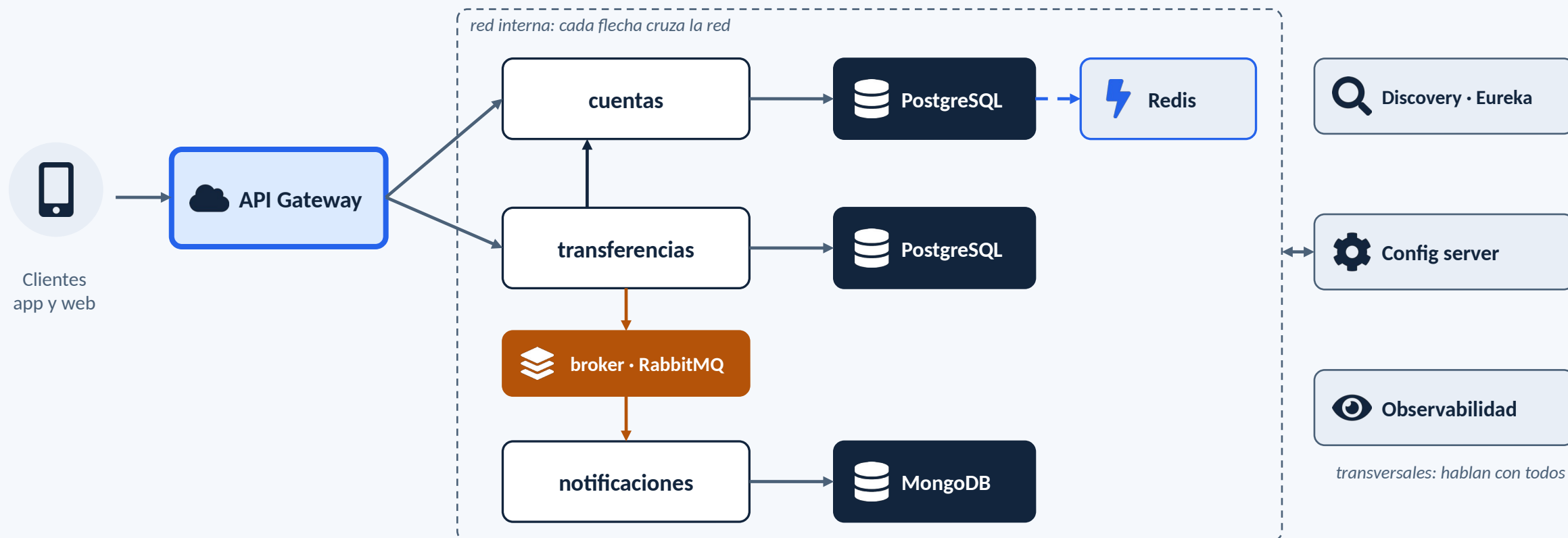
¿Podrían hacer público su repo ahora mismo sin comprometer una credencial?

Después: las dos más votadas de la Discussion #25 (12-factor y Borg).

Partir el monolito crea cinco problemas nuevos y uno de visibilidad



API Gateway: una sola puerta, políticas una sola vez



Problema que resuelve

Sin gateway cada servicio expone su puerto: TLS, identidad y límites se programan N veces y el cliente se acopla a la topología.



Dato duro

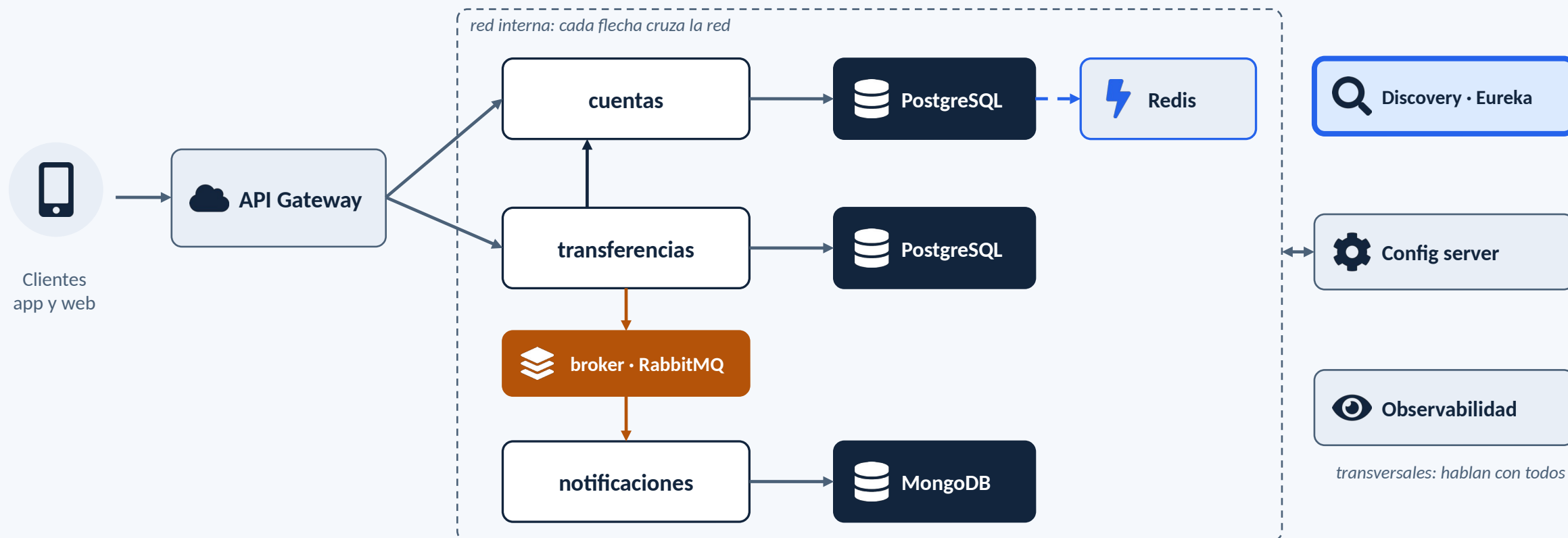
Spring Cloud Gateway 5 (nov-2025). Kubernetes Gateway API es GA desde el 31-oct-2023; Ingress quedó congelado.



Incidente real

Cloudflare, 18-nov-2025: un archivo creció más de lo previsto, el proxy central hizo panic y devolvió 5xx de 11:28 a 17:06 UTC.

Service discovery: el nombre es estable, la dirección no



Problema que resuelve

En contenedores la IP cambia en cada arranque y al escalar hay N instancias: el nombre (naming) se separa de la dirección (binding).



Dato duro

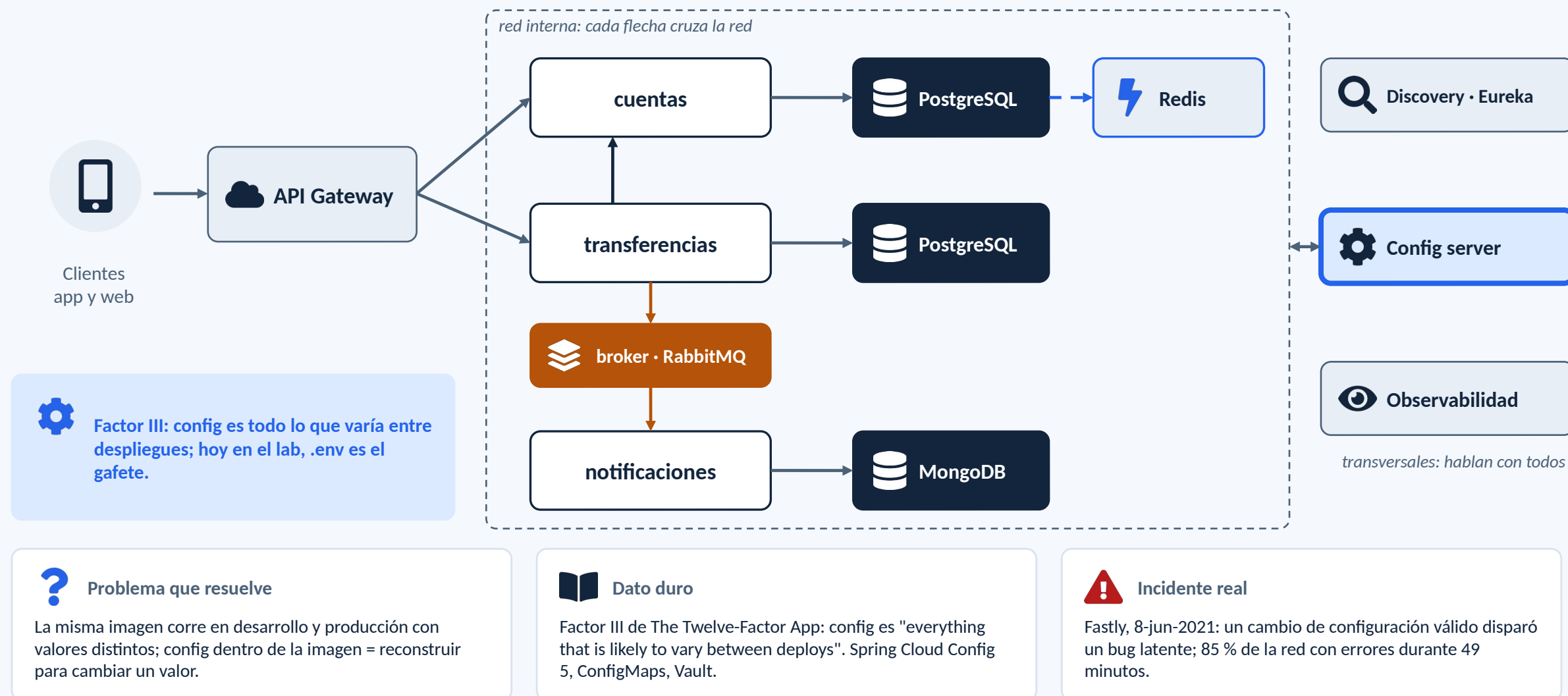
Eureka: heartbeat cada 30 s, expulsión a los 90 s. En Kubernetes lo hacen Service + CoreDNS. La JVM cachea DNS 30 s.



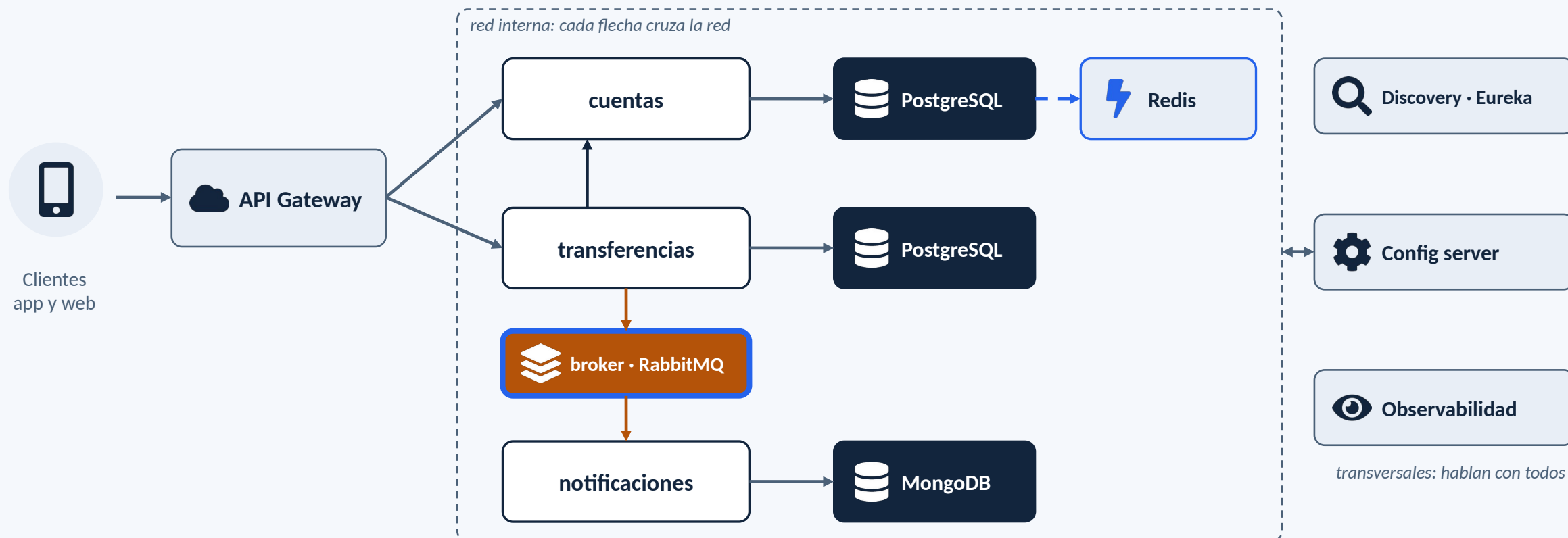
Incidente real

Roblox, 28 al 31-oct-2021: Consul degeneró bajo carga, los servicios dejaron de encontrarse y la telemetría dependía de Consul. 73 horas.

Config server: la imagen es el uniforme, la configuración es el gafete



Broker: de "procesar ahora" a "procesar eventualmente"



? Problema que resuelve

Una llamada síncrona acopla disponibilidad: si notificaciones está caído, la transferencia falla. El broker guarda el mensaje para después.

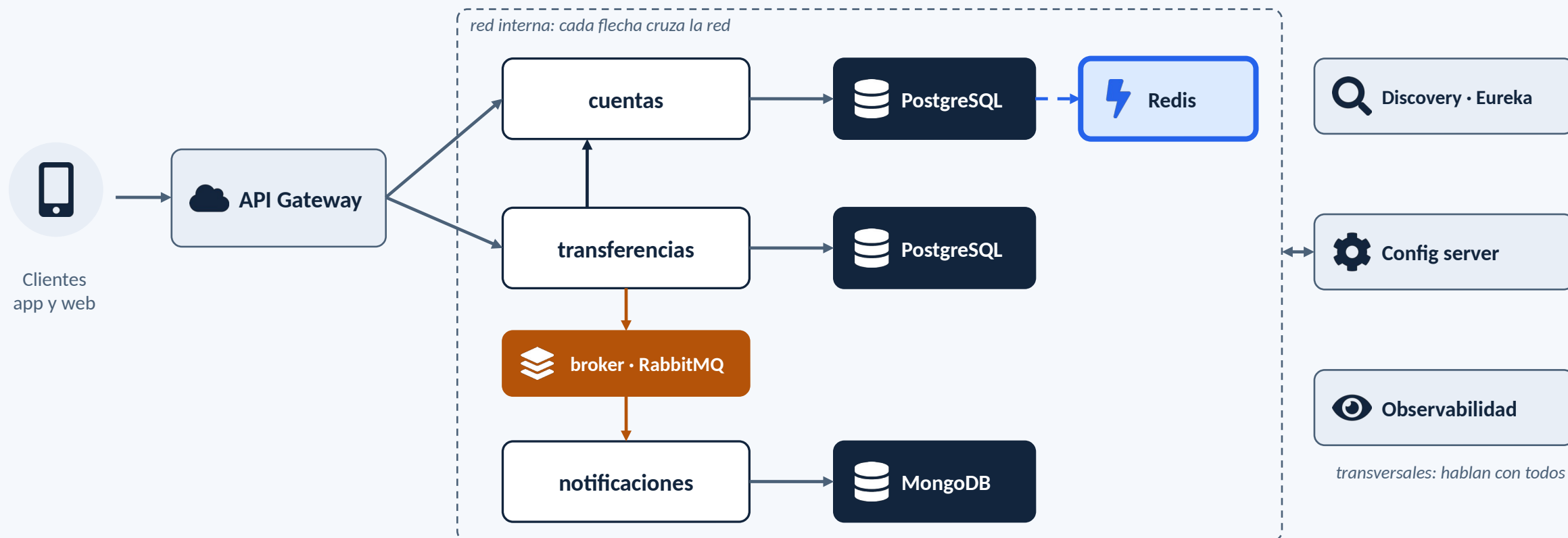
📖 Dato duro

At-least-once: ack tras persistir; sin ack, reentrega; N fallos, dead-letter queue. RabbitMQ 4.0 (sep-2024), Kafka 4.0 sin ZooKeeper (mar-2025).

⚠️ Incidente real

AWS Kinesis, 25-nov-2020: Cognito escribía al broker de forma bloqueante; con el broker caído, el servicio "asíncrono" cayó también.

Caché: una copia que se permite estar vieja



? Problema que resuelve

La base de datos es el recurso caro y compartido. Cache-aside: si la llave está, hit; si no, miss: se lee de la base y se guarda con TTL.

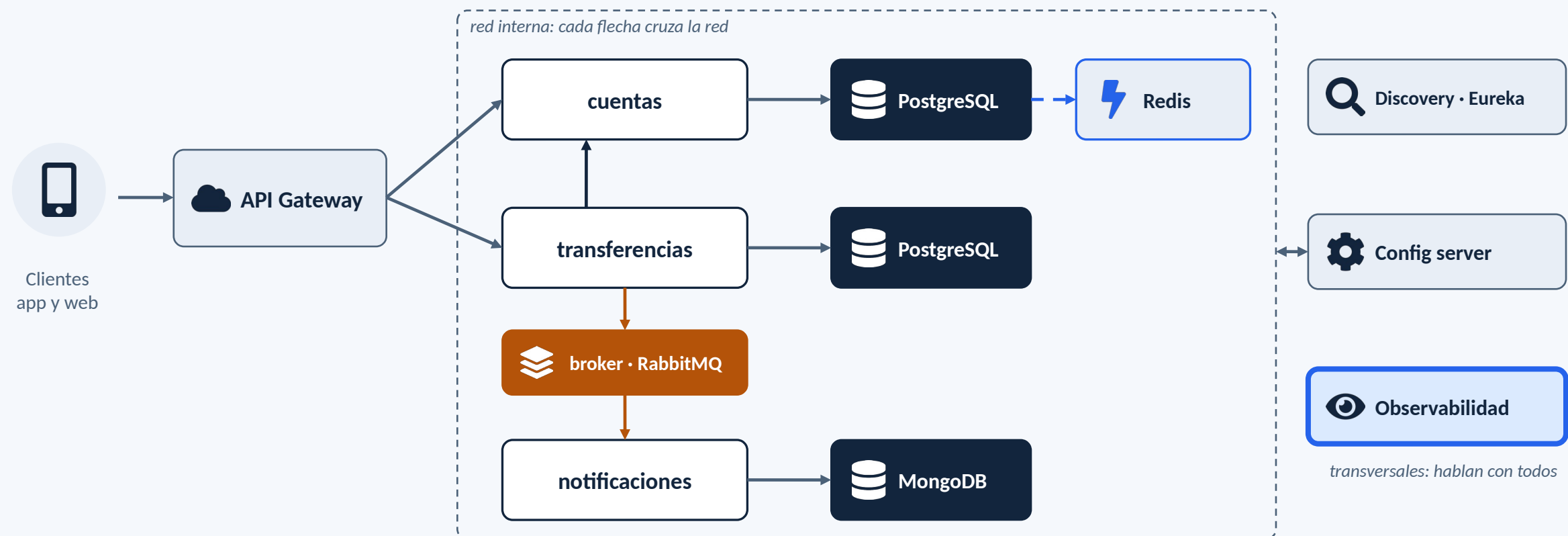
📖 Dato duro

De 99 % a 98 % de aciertos se duplica el tráfico a la base. Lo difícil es invalidar. Redis 8 (may-2025, AGPLv3) y Valkey (2024).

⚠️ Incidente real

Slack, 22-feb-2022: nodos de Memcached sustituidos por instancias vacías en el pico; la base hizo timeout y la caché no se podía rellenar.

Observabilidad: sin trace id hay veinte bitácoras y ninguna historia



? Problema que resuelve

Cuando una operación cruza varios servicios, ¿dónde falló y dónde tardó? Logs (qué pasó aquí), métricas (cómo se comporta), trazas (el recorrido).

📖 Dato duro

W3C traceparent = versión, trace id, parent id, flags; viaja en cada llamada. OpenTelemetry con las tres señales estables; Jaeger v2 (nov-2024).

⚠ Incidente real

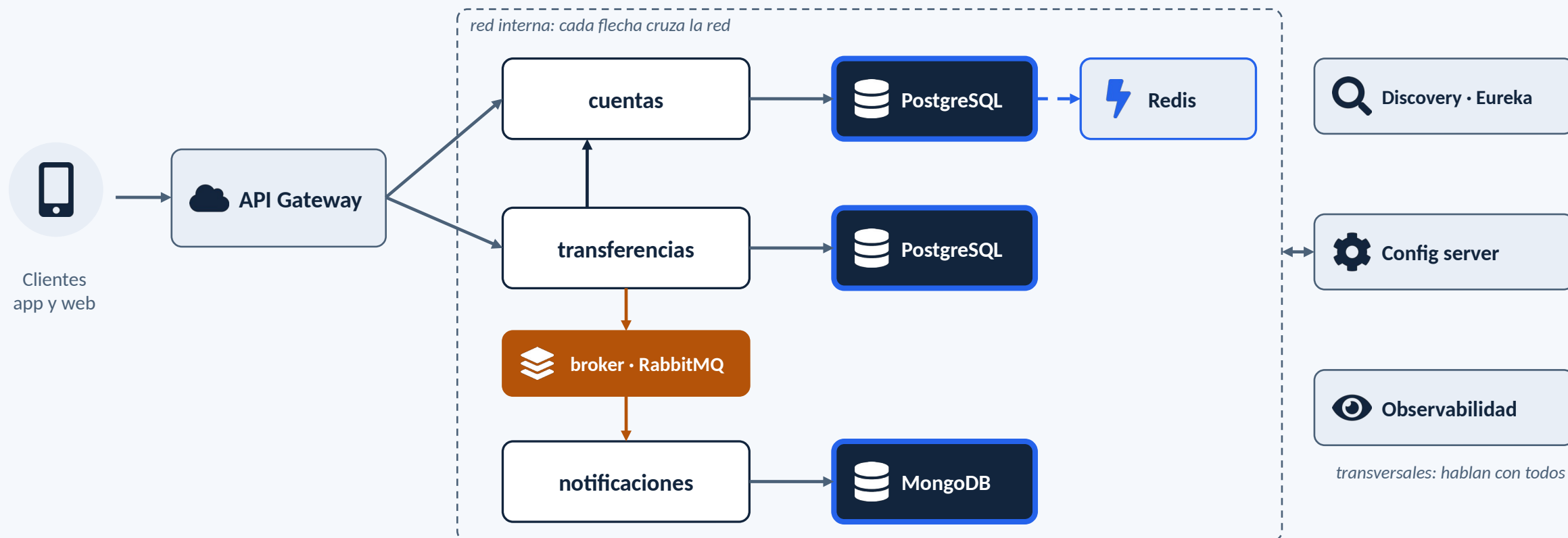
El monitoreo que depende de lo que monitorea: la telemetría de Roblox dependía de Consul; CloudWatch dependía de Kinesis (2020).

12-factor: cuatro factores que hoy ya vieron en las piezas



12factor.net: Wiggins, Heroku, 2011. Los otros ocho: el lab de hoy y la S5.

Cada servicio es dueño de su almacén; se elige por patrón de acceso



La regla

Los datos de un servicio se leen por su API o sus eventos, nunca entrando a la base de datos ajena.



Dato duro

Fowler acuñó polyglot persistence el 16-nov-2011: primero cómo se manipulan los datos, después la tecnología.



Lo que se pierde

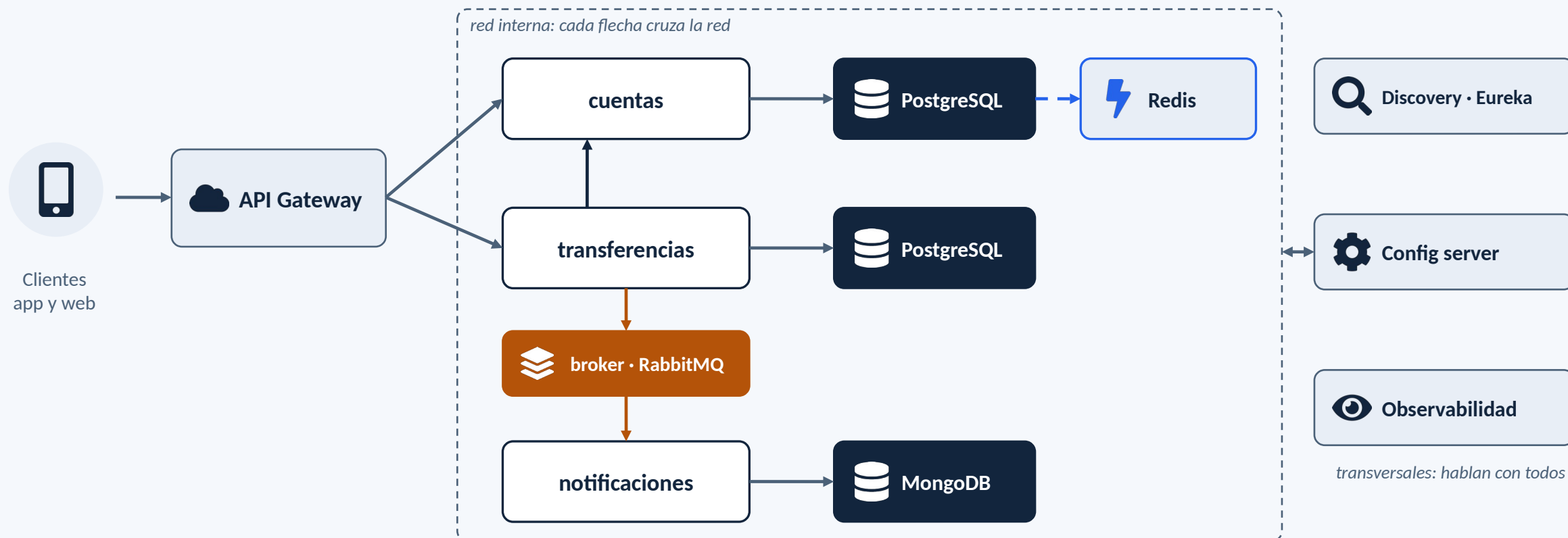
No hay joins ni transacciones entre servicios: la consistencia se vuelve diseño (S6, S12); hacen falta eventos e idempotencia.

Seis familias, seis preguntas: el criterio es cómo se lee y cómo se escribe

Familia	Fortaleza	Caso típico	Ejemplo · AWS
Relacional	ACID, joins, integridad (UNIQUE)	cuentas, transferencias, libro mayor	PostgreSQL · Aurora, RDS
Documental	el agregado completo en una lectura, esquema flexible	expediente de cliente, catálogo	MongoDB · DocumentDB
Llave-valor	acceso por llave en microsegundos	caché, sesiones, contadores	Redis, Valkey · DynamoDB, ElastiCache
Columnar	agregaciones sobre muchas filas y pocas columnas	analítica, series de tiempo	Cassandra · Redshift, Keyspaces
Grafo	recorrer relaciones de profundidad variable	fraude, recomendaciones, avales	Neo4j · Neptune
Objetos	archivos grandes e inmutables por URL	documentos, imágenes, respaldos	MinIO · S3

En el sistema de referencia conviven tres: PostgreSQL (transaccional), MongoDB (documentos) y Redis (llave-valor). Profundización en la S9: replicación, sharding, CRDTs.

El mapa completo: seis piezas, un problema cada una



Gateway: una puerta

Discovery: nombre estable

Config: fuera de la imagen

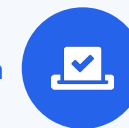
Broker: al menos una vez

Caché: copia con TTL

Observabilidad: trace id

La decisión de hoy • FarmaYa: ¿reintentar o no?

ritual • 12 min



? El caso (ajeno, ensayo, no se califica)

FarmaYa vende medicinas con entrega a domicilio. Al pagar, la app llama a POST /pedidos/{id}/pagar, que llama a la pasarela de tarjetas. La pasarela tarda de 1 a 8 s y a veces no responde. El timeout de la app es de 3 s. Se venció. ¿Qué hace la app?

A • Reintentar ya

at-least-once

Persigue: ningún pedido pagado se pierde.
Exige: Idempotency-Key en la pasarela.
Sacrifica: latencia y depender de que la pasarela deduplique de verdad.

B • No reintentar

at-most-once

Persigue: nunca cobrar dos veces.
Pide al cliente revisar su estado.
Sacrifica: pedidos en limbo y soporte manual.

C • Evento al broker

"pago solicitado" y responder "en proceso"

Persigue: la app disponible aunque la pasarela esté lenta.
Exige: consumidor idempotente y notificar el resultado.
Sacrifica: confirmación inmediata.

elección: A | B | C porque: el requisito que persigo sacrificio: lo que dejo atrás

Mismo formato de tres líneas que la Decisión 0. Los 10 equipos deciden en el chat; dos defienden en voz.

Para la S5: consenso, y el sistema desglosado con compose



Lectura 4 • para la S5

Ongaro y Ousterhout, "In Search of an Understandable Consensus Algorithm" (Raft, 2014).

Secciones 1 a 5.



Laboratorio de la S5

El sistema del curso desglosado con compose: cómo se hablan los contenedores; gateway, discovery, config, RabbitMQ, Redis y Jaeger en vivo.

Al final, Kubernetes básico.



Requisitos

El lab de hoy en entregas/apellido_nombre/s04/, push antes del domingo.

Pensar con quién quieren equipo: los equipos de 4 se forman en la S5.

sábado 19 • 06:59

la lectura, en tiempo; tarde hasta el domingo

2 preguntas + 1 hallazgo en la Discussion

Ocho preguntas que hoy ya pueden responder

1

¿Qué tres historias esconde un timeout y cuál es la única salida segura?

2

¿Qué diferencia hay entre at-least-once y exactly-once como efecto?

3

¿Por qué DELETE es idempotente aunque responda 204 y luego 404?

4

¿Por qué un Map en memoria no basta para las claves de idempotencia?

5

¿Qué problema resuelve el API gateway y qué pasa si él falla?

6

¿Cómo encuentra un servicio a otro si las IPs cambian, y cada cuánto late Eureka?

7

¿Cuándo conviene un broker en lugar de una llamada directa, y qué se sacrifica?

8

¿Con qué criterio se elige el tipo de base de datos de cada servicio?

La que no puedan responder, esa es la que se estudia esta semana. Están respondidas en la guía.

Fuentes de la sesión

- Waldo, Wyant, Wollrath y Kendall. A Note on Distributed Computing. Sun Microsystems, SMLI TR-94-29, 1994 (la Lectura 2).
- Birrell y Nelson. Implementing Remote Procedure Calls. ACM TOCS 2(1), 1984.
- Fielding. Architectural Styles and the Design of Network-based Software Architectures, cap. 5. UC Irvine, 2000.
- RFC 9110, HTTP Semantics (IETF, 2022), §9.2.1 y §9.2.2 • RFC 10008, The HTTP QUERY Method (IETF, 2026).
- Stripe. Idempotent requests (docs.stripe.com) • Leach, Designing robust and predictable APIs with idempotency, 2017.
- gRPC. Google, anuncio del 26 de febrero de 2015; grpc.io, gRPC over HTTP/2 • Protocol Buffers, Encoding (protobuf.dev).
- Wiggins. The Twelve-Factor App (12factor.net), 2011 • Burns, Grant, Oppenheimer, Brewer y Wilkes. Borg, Omega, and Kubernetes. ACM Queue 14(1), 2016 (la Lectura 3).
- Ongaro y Ousterhout. In Search of an Understandable Consensus Algorithm (Raft). USENIX ATC, 2014 (la Lectura 4).
- AWS. Summary of the Amazon Kinesis Event in US-EAST-1, 25-nov-2020 • Brooker, Exponential Backoff And Jitter, 2015.
- Roblox, Return to Service 10/28-10/31 2021 • Cloudflare, outage on November 18, 2025 • Slack, Incident on 2-22-22 • Fastly, Summary of June 8 outage, 2021 • Twilio, Billing Incident Post-Mortem, 2013.
- Spring Boot 4.1, Spring Cloud 2025.1 (Gateway, Netflix, Config), Spring AMQP 4.0: documentación de referencia en docs.spring.io.
- Fowler. PolyglotPersistence, 2011 • Newman. Building Microservices, 2.^a ed., 2021 • Kleppmann. Designing Data-Intensive Applications, 2017.
- Sigelman et al. Dapper, Google 2010 • W3C Trace Context Level 1 • OpenTelemetry specification status.

Enlaces completos y numerados en 01-investigacion.md de la S03 (103 fuentes) y en lecturas/README.md del repo del curso.